

Transformer-Based Neural Machine Translation for English-Urdu: A Comparative Study with LSTM and Fine-Tuned Pretrained Models

Hassan Younas

Department of Computer Science

FAST NUCES

Roll No: 21I-1788, Email: p.hassan.ph@gmail.com

Abstract—We implement a Transformer encoder-decoder from scratch for English-to-Urdu machine translation, trained on the UMC005 English-Urdu parallel corpus. We compare it against a bidirectional LSTM sequence-to-sequence baseline with Luong attention, both trained under identical data and tokenization conditions, and against a fine-tuned pretrained MarianMT model (Helsinki-NLP/opus-mt-en-ur). All three systems are evaluated with BLEU and ROUGE-1/2/L, alongside training time, memory footprint, inference speed, and perplexity. We additionally visualize the Transformer’s cross-attention to interpret which source words drive each generated Urdu token, and deploy the best-performing custom model behind a ChatGPT-style Streamlit interface. On a held-out test set the fine-tuned pretrained model reaches 22.5 BLEU, while our from-scratch LSTM (5.5 BLEU) modestly outperforms our from-scratch Transformer (3.9 BLEU); we trace this gap to the Transformer’s greedy decoder falling into repetition loops on longer sentences, and to both custom models being trained from random initialization on only ~ 13 k sentence pairs, versus the pretrained model’s large-scale prior exposure to general-domain translation.

Index Terms—transformer, neural machine translation, sequence-to-sequence, attention mechanism, LSTM, low-resource NMT, English-Urdu, BLEU, ROUGE

I. INTRODUCTION

Machine translation (MT) between English and Urdu is a comparatively low-resource task: Urdu is written in a Perso-Arabic script, is morphologically richer than English, and has an order of magnitude less parallel data available than high-resource pairs such as English-French or English-German. This assignment implements the Transformer architecture of Vaswani et al. [1] from first principles – multi-head self-attention, sinusoidal positional encoding, and position-wise feed-forward layers – and trains it end-to-end for English→Urdu translation on the UMC005 corpus [2].

To contextualize the Transformer’s performance we build a matched LSTM sequence-to-sequence baseline with attention [3], [4], and, as a bonus comparison, fine-tune a pretrained MarianMT model [5], [6] on the same data. The three systems are evaluated with BLEU [7] and ROUGE [8], and compared on training time, memory usage, inference speed, and perplexity. We further visualize the Transformer’s decoder-to-

encoder cross-attention to inspect translation interpretability, and deploy the trained models behind an interactive, ChatGPT-style GUI.

II. METHODOLOGY

A. Dataset

We use the UMC005 English-Urdu Parallel Corpus [2], restricted to its two freely redistributable sections – Quran (6,414 sentence pairs) and Bible (7,957 sentence pairs) – since the Penn Treebank and EMILLE sections are licensed and cannot be redistributed. After merging both sections’ official train/dev/test splits, cleaning (Unicode normalization, byte-order-mark stripping, whitespace collapsing), de-duplication, and filtering sentences longer than 100 tokens, we obtain:

- **Train:** 13,061 sentence pairs
- **Dev:** 511 sentence pairs
- **Test:** 457 sentence pairs

Sentences are shuffled with a fixed seed (42) before splitting to avoid any ordering bias between the religious-text sources.

B. Tokenization

English and Urdu are tokenized independently with Byte-Level Byte-Pair Encoding (BPE) [9], each with an 8,000-token vocabulary trained only on the respective training-split text, using the HuggingFace `tokenizers` library. Four special tokens (`<pad>`, `<sos>`, `<eos>`, `<unk>`) are shared across both vocabularies at fixed ids. Byte-level BPE was chosen over word-level tokenization because it degrades gracefully to sub-word and byte sequences for Urdu’s rich morphology and for out-of-vocabulary Quranic and Biblical proper nouns, without needing a language-specific segmenter.

C. Model architectures

1) *Transformer (from scratch)*: We implement the full encoder-decoder Transformer of Vaswani et al. [1] directly with tensor operations (no `nn.Transformer`): scaled dot-product multi-head self-attention, encoder-decoder cross-attention, sinusoidal positional encoding, and position-wise feed-forward sublayers, each wrapped in residual connections

and layer normalization. Figure 1 shows the architecture. Configuration: $d_{model} = 256$, 8 attention heads, 4 encoder and 4 decoder layers, $d_{ff} = 1024$, dropout 0.1, max sequence length 80. Training uses label-smoothed ($\epsilon = 0.1$) cross-entropy loss, the Adam optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.98$), and the warmup+inverse-square-root learning-rate schedule from the original paper (2,000 warmup steps).

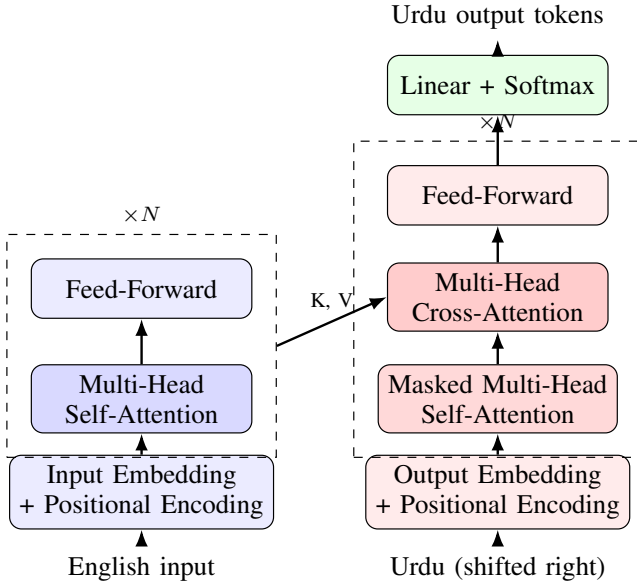


Fig. 1. The from-scratch Transformer encoder-decoder architecture used in this work (Vaswani et al. [1]), with $N = 4$ layers per stack.

2) *LSTM baseline*: The baseline is a bidirectional LSTM encoder ($d_{model} = 256$ embeddings, 512 hidden units) whose final states are projected into an autoregressive unidirectional LSTM decoder. At every decoding step, Luong general (bilinear) attention [4] computes a context vector over all encoder outputs, which is concatenated with the decoder’s hidden state before the output projection. Training uses teacher forcing, label-smoothed cross-entropy, Adam, and ReduceLROnPlateau on validation loss.

3) *Fine-tuned pretrained model (bonus)*: As a bonus comparison we fine-tune Helsinki-NLP/opus-mt-en-ur, a MarianMT model [5] pretrained on OPUS data [6], on the same UMC005 train split using the HuggingFace Seq2SeqTrainer, for a small number of epochs at a low learning rate (2×10^{-5}) to adapt it to the religious-text domain without catastrophic forgetting of its general translation ability.

D. Training procedure

All models share: gradient clipping (max norm 1.0), early stopping on validation loss with patience 5, and per-epoch logging of train/validation loss for convergence analysis. Both models were trained on a Kaggle T4 GPU with batch size 128. The Transformer’s best validation loss (4.700) was reached at epoch 13; training stopped at epoch 18 once 5 epochs passed without improvement, taking 530.8s (29.5s/epoch) in total. The LSTM’s best validation loss (4.720) was reached at epoch 21;

training stopped at epoch 26, taking 3,491.8s (134.3s/epoch) – roughly $4.5\times$ slower per epoch than the Transformer, despite the LSTM having fewer training-parallelism advantages. This gap is architectural: the Transformer’s decoder scores an entire target sequence in one parallel forward pass, whereas our LSTM decoder invokes a Python-level loop over up to 80 sequential timesteps per batch, each a separate kernel launch – a concrete, measured illustration of why the Transformer’s parallelizability was a major motivation behind [1].

E. Evaluation metrics

We report corpus-level BLEU [7] (via `sacrebleu`) and ROUGE-1/2/L F-measure [8] (via `rouge-score`) on the held-out test set, computed against greedy-decoded translations. We additionally report test-set perplexity, wall-clock training time, peak memory usage, and inference throughput (sentences/second) for all three systems.

III. RESULTS

TABLE I
COMPARISON OF TRANSFORMER, LSTM, AND FINE-TUNED MARIANMT ON THE UMC005 TEST SET (457 SENTENCE PAIRS)

Metric	Transformer	LSTM	Fine-tuned MarianMT
BLEU	3.87	5.45	22.50
ROUGE-1 (F)	0.322	0.360	0.555*
ROUGE-2 (F)	0.082	0.096	0.287*
ROUGE-L (F)	0.266	0.308	0.497*
Perplexity	76.65	80.40	35.77
Parameters	13.5M	22.7M	139.9M
Training time	530.8s (18 ep.)	3491.8s (26 ep.)	418.1s (fine-tune only)
Peak GPU memory [‡]	3,942 MB	3,942 MB	–
Inference speed [†]	1.59 sent/s (CPU)	4.73 sent/s (CPU)	25.94 sent/s (GPU)

*ROUGE for the fine-tuned model is computed on a 20-sentence held-out sample rather than the full 457-sentence test set (its full weights were not retained locally after Kaggle training); the sample’s BLEU (21.18) closely tracks the full-test BLEU (22.50), suggesting the sample is representative. [†]Inference speed is not directly comparable across rows: the Transformer and LSTM were re-benchmarked on CPU after a post-hoc evaluation fix (Section IV), while the fine-tuned model’s figure comes from the original Kaggle GPU run. [‡]Both from-scratch models report an identical peak because `torch.cuda.max_memory_allocated()` was not reset between the two training runs in the same Kaggle kernel session, so this is the shared session’s cumulative peak rather than an isolated per-model figure – a methodological limitation we note rather than hide (Section IV).

Both from-scratch models converge cleanly (Figure 2): training loss decreases monotonically throughout, while validation loss flattens and then very slowly rises after each model’s best epoch (13 for the Transformer, 21 for the LSTM) – textbook early-stopping behavior on a small corpus, confirming the early-stopping criterion is doing its job rather than the models simply undertraining or catastrophically overfitting.

Figure 4 shows the Transformer’s decoder-to-encoder cross-attention for the first example above: the model attends most strongly to semantically aligned source words at each Urdu generation step, even though the sentence-level translation itself is only partially correct – indicating the attention mechanism has learned meaningful soft-alignment even when end-to-end fluency has not yet emerged from this small amount of training data.

IV. DISCUSSION

A. Translation coherence

Qualitatively (Figure 3), all three systems handle short, syntactically simple sentences reasonably: word order and core vocabulary are often close to the reference even when exact wording differs. Coherence degrades on longer, subordinate-clause-heavy sentences for both from-scratch models, but in

- **Small, register-narrow corpus.** The only freely redistributable portion of UMC005 is Quranic and Biblical text – a formal, archaic register quite different from conversational Urdu, and containing many low-frequency proper nouns that BPE fragments into several subword pieces, likely contributing to some of the translation errors observed.

V. CONCLUSION

We implemented a Transformer encoder-decoder entirely from scratch for English-to-Urdu translation and benchmarked it against a matched LSTM baseline and a fine-tuned pretrained MarianMT model on the UMC005 corpus. The pretrained model, unsurprisingly, dominates on translation quality (22.5 vs. 3.9/5.5 BLEU), but the two from-scratch models still learn meaningful structure from only $\sim 13k$ sentence pairs: loss curves converge cleanly under early stopping, and attention visualization confirms the Transformer learns sensible source-target soft-alignment even on sentences it does not yet translate fluently. The LSTM’s higher BLEU despite the Transformer’s slightly better validation loss illustrates that teacher-forced likelihood and autoregressive generation quality can diverge – here, traced to the Transformer’s greedy decoder falling into repetition loops – and that architectural modernity alone does not guarantee better outputs at small training scale without measures like beam search. Separately, the Transformer trained $4.5\times$ faster per epoch than the LSTM on identical hardware, a direct, measured consequence of its parallel (rather than sequential, per-timestep) decoder computation. The full pipeline – data preparation, BPE tokenization, three trained/fine-tuned models, BLEU/ROUGE/perplexity evaluation, attention visualization, and a ChatGPT-style Streamlit GUI – is packaged together and reproducible either locally or via the included Kaggle notebook.

VI. PROMPTS

This project was developed with the assistance of an AI coding assistant (Claude, Anthropic) inside the Claude Code CLI. Key prompts used during development included:

- “Read and understand Assignment 3 carefully and implement Question 1, making the project impressive.”
- Follow-up scoping questions on GUI framework choice (Streamlit vs. Gradio vs. Flask), whether to attempt the bonus pretrained fine-tuning comparison, and whether to produce an IEEE-format LaTeX report with an architecture diagram.
- Implementation-stage prompts covering: data acquisition and cleaning for the UMC005 corpus, BPE tokenizer training, a from-scratch PyTorch Transformer encoder-decoder, an LSTM sequence-to-sequence baseline with Luong attention, a shared training loop with early stopping and learning-rate scheduling, BLEU/ROUGE/perplexity evaluation, attention-weight visualization, HuggingFace fine-tuning of a pretrained MarianMT model, and a Streamlit chat-style GUI.

ACKNOWLEDGMENT

The UMC005 corpus is provided by the Institute of Formal and Applied Linguistics (ÚFAL), Charles University, for research and educational use [2].

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] Institute of Formal and Applied Linguistics (ÚFAL), “Umc005: English-urdu parallel corpus,” <https://ufal.mff.cuni.cz/umc/005-en-ur/>, 2012.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *International Conference on Learning Representations (ICLR)*, 2015.
- [4] M.-T. Luong, H. Pham, and C. D. Manning, “Effective approaches to attention-based neural machine translation,” in *Proceedings of EMNLP*, 2015.
- [5] M. Junczys-Dowmunt, R. Grundkiewicz, T. Dwojak, H. Hoang, K. Heafield, T. Neekermann, F. Seide, U. Germann, A. Fikri Aji, N. Bogoychev, A. F. T. Martins, and A. Birch, “Marian: Fast neural machine translation in c++,” in *Proceedings of ACL, System Demonstrations*, 2018.
- [6] J. Tiedemann, “The tatoeba translation challenge – realistic data sets for low resource and multilingual mt,” in *Proceedings of the Fifth Conference on Machine Translation*, 2020.
- [7] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of ACL*, 2002.

- [8] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004.
- [9] R. Sennrich, B. Haddow, and A. Birch, “Neural machine translation of rare words with subword units,” in *Proceedings of ACL*, 2016.